

day 11 | 250922 M

Today we will continue with integration, but we will flip the script and you will work together to implement Simpson's Rule and do exercises 5.2 and 5.3. Simpson's Rule is based on fitting a parabola to a portion of the curve we are interested in and then iterating over each parabola. So look at equation 5.9 on page 146 in the textbook and figure out how to pull that off in python using the `numpy.sum` function.

Also while we are here I want to emphasize something to you. Look at the author's comment after the problem on page 148. He says, "Note that there is no known way to perform this particular integral analytically, so numerical approaches are the only way forward." This is a very important point and one worth keeping in mind as we go through this class. We are doing things that are increasingly impossible to be done any other way. So computational physics is often not a short cut or a way out of certain problems, but rather it is the only way to get a particular answer.

```
In [147... import numpy as np
import matplotlib.pyplot as plt
```

```
In [139... def trap(func, a, b, steps):
    dx = (b-a)/steps
    x = np.linspace(a, b, steps+1)
    y = func(x)
    s = dx/2*(func(a)+func(b))
    s = s + np.sum(y[1:steps]*dx)
    return(s)
```

```
In [9]: t = np.array([11, 14, 1, -9, 30, 28, 50])
```

```
In [22]: t[3::2]
```

```
Out[22]: array([-9, 28])
```

```
In [159... def simp1(func, a, b, steps):
    if steps%2!=0:
        steps = steps + 1
        print('i added one step')
    dx = (b-a)/steps
    x = np.linspace(a, b, steps+1)
    y = func(x)
    s = dx/3*(func(a)+func(b))
    s = s + dx/3*(4*np.sum(y[1:steps:2]) + 2*np.sum(y[2:steps-1:2]))
    return(s)
```

```
In [160... def simp2(func, a, b, steps):
    if steps%2==0:
        steps = steps + 1
```

```

        print('i added one step')
    dx = (b-a)/steps
    x = np.linspace(a, b, steps)
    y = func(x)
    s = dx/3*(func(a)+func(b))
    s = s + dx/3*(4*np.sum(y[1:steps-1:2]) + 2*np.sum(y[2:steps-2:2]))
    return(s)

```

```

In [168... def simp3(function,start,stop,steps):
    if steps%2 == 0:
        steps = steps + 1
    h = (stop-start)/steps
    x = np.linspace(start,stop,steps)
    f = function(x)
    simp = (f[0]+f[steps-1]+4*(f[1]+f[steps-2])+2*np.sum(f[2:steps-2]))*h/3
    return simp

```

```

In [176... def listFromFuncEndpoints(function,x_begin,x_end,steps):
    x_list = np.linspace(x_begin,x_end,steps)
    y_list = function(x_list)
    # old code i don't want to get rid of below
    """
    stepSize = (x_end-x_begin)/float(steps)
    for i in range(steps+1):
        x_list.append(x_begin+i*stepSize)
        y_list.append(function(x_begin+i*stepSize))"""
    return x_list, y_list

def IntSimpFromList(x_list, y_list):
    if len(x_list)%2 == 0:
        print('list does not have an odd number of elements')
    else:
        integral = 0.0
        for i in range(1,len(x_list)-1,2):
            integral = integral + (y_list[i-1]+4*y_list[i]+y_list[i+1])*(x_list[i]-x_list[i-1])
        return integral

def simp4(function,start,end,steps):
    #checks for an even number of steps
    if steps%2 == 0:
        steps = steps + 1
        print('I fixed incorrect number of steps by adding one more')

    x_list, y_list = listFromFuncEndpoints(function,start,end,steps)
    integral = IntSimpFromList(x_list,y_list)
    return integral

```

```

In [162... def f(x):
    return(x**4-2*x+1)

```

```

In [173... simp1(f, 0, 2, 10000)

```

```

Out[173... 4.4000000000000002

```

```
In [164...] simp2(f, 0, 2, 10)
```

i added one step

```
Out[164...] 4.00038787878788
```

```
In [172...] simp3(f, 0, 2, 10)
```

```
Out[172...] 3.761454545454546
```

```
In [177...] simp4(f, 0, 2, 10)
```

I fixed incorrect number of steps by adding one more

```
Out[177...] 4.400426666666666
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [151...] simpsons1(f, 0, 2, 10)
```

```
Out[151...] 4.400426666666667
```

```
In [152...] simpsons1(f, 0, 2, 100)
```

```
Out[152...] 4.400000042666668
```

```
In [153...] simpsons1(f, 0, 2, 1000)
```

```
Out[153...] 4.400000000004267
```

```
In [ ]:
```

```
In [ ]:
```

```
In [156...] simpsons2(f,0,2,11)
```

```
Out[156...] 4.00038787878788
```

```
In [140...] trap(f, 0, 2, 10)
```

```
Out[140...] 4.506560000000001
```

```
In [141]: trap(f, 0, 2, 100)
```

```
Out[141]: 4.401066656
```

```
In [145]: trap(f, 0, 2, 1000)
```

```
Out[145]: 4.400010666665601
```

```
In [ ]: trap(f, 0, 2, 10000)
```

```
In [ ]:
```

```
In [98]: def func(x): return(f(x))
a = 0
b = 2
steps = 11

s = 0

dx = (b-a)/steps
x = np.linspace(a, b, steps)
print(x)
y = func(x)
s = dx/3*(func(a)+func(b))
s = s + 4*np.sum(y[1:steps-1:2])*dx/3 + 2*np.sum(y[2:steps-2:2])*dx/3
```

```
[0.  0.2 0.4 0.6 0.8 1.  1.2 1.4 1.6 1.8 2. ]
```

```
In [77]: np.linspace(0,2,10+1)
```

```
Out[77]: array([0. , 0.2, 0.4, 0.6, 0.8, 1. , 1.2, 1.4, 1.6, 1.8, 2. ])
```

```
In [99]: s
```

```
Out[99]: 4.00038787878788
```

```
In [ ]:
```